BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure
E – Environment
A – Actuators
S – Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The variables representing the physical environment typically include:

self.grid_width – width of the grid
self.grid_height – height of the grid
self.walls – locations of walls/obstacles
self.food_positions (or similar) – locations of food/items
self.opponents – positions or information about opponent agents

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Answer: Multi-Agent

Justification: The presence of self.opponents indicates that other agents exist in the environment. Since the main agent interacts with or is affected by these opponents, the environment is classified as multi-agent.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts (observations) an agent has received from the environment since it began operating.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Answer: Sensors (S)

Explanation: The get_percept() method provides the agent with information about the environment, functioning as the agent's sensing mechanism.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Answer: Fully Observable (assuming the returned dictionary includes all relevant environment information such as agent position, walls, food, opponents, and other necessary state variables).

Explanation: The environment is fully observable because the agent receives all the information needed to know the current state of the environment. There is no hidden information.

If the dictionary only contains nearby cells or limited information instead of the complete environment, then it would be partially observable.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Answer: Performance Measure (P)

Explanation: Deducting points changes how well the agent is performing according to the defined evaluation criteria.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The performance measure should evaluate the results of the agent's actions in the environment, not how much computation the agent performs internally. Hitting a wall is an external outcome that reflects ineffective behavior and affects task success. Measuring environmental outcomes ensures that the agent is rewarded or penalized based on achieving its goals, while allowing different algorithms or implementations to solve the task without being judged solely on their internal processing effort.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes partially observable. Although the traps exist in the environment state (self.toxic_traps), the agent cannot directly perceive their locations through its sensors. As a result, the agent must make decisions without complete information about the environment.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The 'smells_toxin' sensor provides additional information about the environment by indicating whether the agent is currently on a toxic trap. This expands the agent's percept sequence and enables it to make better-informed decisions. A rational agent uses this information to avoid trap locations, reducing penalties and increasing its expected score (utility). Better percepts lead to better action choices under the performance measure.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit is an agent that keeps cleaning an already clean square (or repeatedly performs unnecessary actions) because the performance measure rewards cleaning actions rather than rewarding the actual goal of maintaining a clean environment. This demonstrates that a poorly designed performance measure can encourage unintended behavior, where the agent maximizes its score without accomplishing the intended objective.

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING